AURA-HOI: High-Occupancy Itemset Mining via Equivalence Classes and Residual Envelope Pruning

Quan Van[1*]

[1*]Independent Researcher, Ho Chi Minh City, Vietnam.

Corresponding author(s). E-mail(s): vanhaminhquan2406@gmail.com;

Abstract

Occupancy measures how much of its supporting transactions an itemset covers. Because occupancy is not anti-monotone, exact high-occupancy itemset (HOI) miners rely on upper bounds to prune the search space. The bound used by existing miners depends only on the lengths of the supporting transactions; on databases whose transactions all have the same length it degenerates to the support, so these miners must enumerate every frequent itemset. We present AURA-HOI, an exact depth-first miner over prefix equivalence classes of transaction-ID lists. Its central component is a length-capped residual bound that combines transaction lengths with the largest itemset that can still be generated in the current branch. We prove that the bound is safe, that it is never weaker than the transaction-length bound or the purely residual bound, and that the miner returns exactly the set of HOIs. The bound is computed in the same pass as the TID-list intersection, and in constant time when transaction lengths are uniform. We also show that a natural shortcut, taking the residual length from the later siblings only, loses results. In single-threaded experiments on ?? public benchmarks and ?? threshold settings, AURA-HOI returns exactly the same itemsets as two exact baselines, HEP and DFHOI, and agrees with an exact rational-arithmetic oracle on ?? randomized test cases. It is faster than DFHOI in ?? of the ?? settings, with a geometric-mean speedup of ??× and a maximum of ??× on dense data, where it also performs far fewer candidate intersections. On sparse data its speedup is small (geometric mean ??×).

Keywords: High-occupancy itemset mining, Pattern mining, Upper bound, Equivalence classes, Vertical mining, Search space pruning

1 Introduction

Frequent itemset mining finds the sets of items that co-occur in at least a given number of transactions [1, 2]. Its efficient algorithms, such as Apriori [2], FP-growth [3] and Eclat [4, 5], rely on the anti-monotonicity of support: every superset of an infrequent itemset is infrequent. Support alone, however, does not say how representative an itemset is of the transactions that contain it. An itemset of two items that occurs in many transactions of length fifty describes only a small part of each of them.

Tang et al. introduced *occupancy* to capture this aspect: the occupancy of an itemset relates its size to the sizes of the transactions that contain it, and patterns with high occupancy are better candidates for recommendation [6]. Deng formulated the mining of high-occupancy itemsets (HOIs) and proposed the HEP algorithm [7], and Nguyen et al. proposed DFHOI, a depth-first algorithm over equivalence classes with early pruning [8]. Occupancy has since been studied with transaction weights [9], in incremental settings [10], with adaptive thresholds [11], as a top-k problem [12], and combined with utility [13–15].

Occupancy is not anti-monotone: adding an item to an itemset increases its size but may shrink its set of supporting transactions, so the occupancy of the extension can be larger or smaller. Exact miners therefore prune with an *upper bound* on the occupancy of all extensions. The bound used by HEP and DFHOI, which we call the transaction-length bound, exploits the fact that an itemset cannot be larger than any transaction that contains it. It depends only on the lengths of the supporting transactions. When all transactions have the same length, as in many categorical benchmarks, this bound equals the support (Proposition 5). It then prunes nothing beyond the support constraint, and the miner enumerates every frequent itemset even when none of them has high occupancy.

A depth-first search over prefix equivalence classes provides a second, independent limit: inside a branch, an itemset can only be extended with the items that remain in its class, so its descendants have bounded size. This is the same principle as the remaining-utility bounds of high-utility itemset mining [16, 17]. This paper combines the two limits in a single bound and builds an exact miner around it. The main contributions are the following.

1. A *length-capped residual bound* on the occupancy of every itemset that can still be generated in a branch (Theorem 3). We prove that it is never weaker than the transaction-length bound or the purely residual bound (Proposition 4), that it is computed in the same linear pass as the TID-list intersection, and that it reduces to a constant-time closed form when transaction lengths are uniform.
2. AURA-HOI, a depth-first miner over prefix equivalence classes of TID-lists that applies the bound at the level of nodes and of whole classes and allocates TID-lists from a stack-like arena. We prove that it returns exactly the set of HOIs (Theorem 6), and we show with a two-transaction counterexample that taking the residual length from the later siblings only, a tempting shortcut, loses results (Remark 1).
3. An evaluation protocol for exact HOI miners: verification against an exact rational-arithmetic oracle on randomized databases, output parity across algorithms,

repeated single-threaded runs without a time limit, memory measured as the increase attributable to the mining phase, an ablation study, a scalability study and a signed-rank test.

Section 2 reviews related work. Section 3 defines the problem. Section 4 develops the bounds and Section 5 the algorithm. Section 6 reports the experiments, and Section 7 concludes.

2 Related work

Frequent itemset mining.

Apriori generates candidates level by level and prunes them with the anti-monotonicity of support [1, 2]. FP-growth avoids candidate generation by recursively projecting a prefix tree [3]. Eclat represents each itemset by the sorted list of identifiers of its supporting transactions (its TID-list), groups itemsets that share a prefix into equivalence classes, and computes supports by intersecting TID-lists [4, 5]; diffsets reduce the size of these lists on dense data [18]. PrePost and PrePost+ use N-lists built from a prefix tree [19, 20]. Condensed representations, such as closed [21–23] and maximal [24] itemsets, reduce the number of reported patterns. Surveys of the field are available in [25–27].

Occupancy.

Tang et al. introduced occupancy as a measure of pattern quality [6]. Deng studied the mining of high-occupancy itemsets and proposed HEP, which stores occupancy lists and prunes with an upper bound derived from transaction lengths [7]. Nguyen et al. proposed DFHOI, which traverses equivalence classes depth-first and applies the bound early [8]. Datta et al. weighted occupancy by transaction occupancy [9]; Kim et al. mined high-occupancy patterns incrementally [10]; Tran et al. proposed a hierarchical set-enumeration tree with an adaptive occupancy threshold that changes from level to level [11]; and Yıldırım addressed the top-k problem [12]. Occupancy has also been combined with utility [13–15]. The adaptive-threshold and top-k formulations return different result sets from the fixed-threshold problem studied here, so we compare with HEP and DFHOI, which solve the same problem.

Bounds for measures that are not anti-monotone.

High-utility itemset mining faces the same difficulty, because utility is not anti-monotone either [28, 29]. HUI-Miner bounds the utility of the extensions of an itemset by its utility plus the remaining utility of the items that can still be appended [16]; FHM adds co-occurrence pruning [30]; and EFIM uses sub-tree and local utility bounds [17]. The residual part of our bound follows the same principle: in a depth-first enumeration, the items that can still be appended are known, which limits what the descendants can achieve. For occupancy, the residual information is the *number* of such items, which we combine with transaction lengths.

Table 1 Running example database with $n = 5$ transactions.

q	t_q	$\lvert t_q\rvert$	$1/\lvert t_q\rvert$
1	$\{A, B, C, D\}$	4	1/4
2	$\{A, B, C\}$	3	1/3
3	$\{A, B, D\}$	3	1/3
4	$\{A, C, D, E\}$	4	1/4
5	$\{B, C, D, E\}$	4	1/4

3 Problem definition

Let I be a finite set of items and $D = \langle t_1, \ldots, t_n \rangle$ a database of n non-empty transactions $t_q \subseteq I$. For an itemset $X \subseteq I$, the set of supporting transactions and the support are

$$T(X) = \{q : X \subseteq t_q\}, \qquad \mathrm{supp}(X) = |T(X)|. \tag{1}$$

We write $|t_q|$ for the length of transaction t_q.

Definition 1 (Occupancy) The (summed) occupancy of an itemset X is

$$\mathrm{occ}(X) = \sum_{q \in T(X)} \frac{|X|}{|t_q|} = |X| \cdot S(X), \qquad S(X) = \sum_{q \in T(X)} \frac{1}{|t_q|}. \tag{2}$$

Each term $|X|/|t_q|$ is the fraction of transaction t_q that X occupies. We call $S(X)$ the reciprocal sum of X.

Definition 2 (High-occupancy itemset) Given a relative threshold $\alpha \in (0, 1]$, let $\xi = \alpha n$. An itemset $X \neq \emptyset$ is a high-occupancy itemset (HOI) if $\mathrm{occ}(X) \geq \xi$. The problem is to find the set $\mathcal{H}$ of all HOIs.

This is the fixed-threshold problem addressed by HEP [7] and DFHOI [8]. Our implementations of both baselines solve exactly this problem, which we verify against an exact oracle in Section 6.3.

Proposition 1 *For every itemset X, $\mathrm{occ}(X) \leq \mathrm{supp}(X)$. Hence every HOI has support at least $\sigma = \lceil \xi \rceil$.*

Proof If $q \in T(X)$ then $X \subseteq t_q$, so $|X| \leq |t_q|$ and each term of (2) is at most 1. There are $\mathrm{supp}(X)$ terms. Since $\mathrm{supp}(X)$ is an integer, $\mathrm{supp}(X) \geq \xi$ implies $\mathrm{supp}(X) \geq \lceil \xi \rceil$. □

Support is anti-monotone, so Proposition 1 gives a first pruning rule: no superset of an itemset with support below σ is an HOI. Occupancy itself is not anti-monotone, as the following example shows.

Example 1 Consider the database of Table 1 with $\alpha = 0.35$, so that $\xi = 7/4$ and $\sigma = 2$. For $X = \{A\}$, $T(X) = \{1, 2, 3, 4\}$ and $\mathrm{occ}(X) = 1 \cdot (1/4 + 1/3 + 1/3 + 1/4) = 7/6 < 7/4$, so $\{A\}$ is not an HOI. Its superset $\{A, B\}$ has $T = \{1, 2, 3\}$ and $\mathrm{occ}(\{A, B\}) = 2 \cdot (1/4 + 1/3 + 1/3) = 11/6 \geq 7/4$, so it is an HOI. An itemset that fails the threshold can therefore have supersets that pass it.

4 Upper bounds on occupancy

All bounds in this section bound the occupancy of every itemset Y that contains a given itemset X and has at most K items. In the search of Section 5, X is the current node and K is the size of the largest itemset that can still be generated from it.

Lemma 2 *If $X \subseteq Y$, then (i) $T(Y) \subseteq T(X)$, and (ii) $|Y| \leq |t_q|$ for every $q \in T(Y)$.*

Proof (i) Every transaction that contains Y contains X. (ii) If $q \in T(Y)$ then $Y \subseteq t_q$. □

For an itemset X, let $\Lambda(X) = \{|t_q| : q \in T(X)\}$ be the set of lengths of its supporting transactions, and for an integer $\lambda \geq 1$ let

$$W_\lambda(X) = \sum_{q \in T(X),\ |t_q| \geq \lambda} \frac{1}{|t_q|} \tag{3}$$

be the reciprocal sum restricted to transactions of length at least λ. Note that $W_1(X) = S(X)$.

Definition 3 (Length-capped residual bound) For an itemset X with $T(X) \neq \emptyset$ and an integer $K \geq 1$,

$$U_K(X) = \max_{\lambda \in \Lambda(X)} \min(\lambda, K) \cdot W_\lambda(X), \tag{4}$$

and $U_K(X) = 0$ if $T(X) = \emptyset$.

Theorem 3 (Safety) *Let $X \subseteq Y$ and $|Y| \leq K$. Then* $\mathrm{occ}(Y) \leq U_K(X)$.

Proof If $T(Y) = \emptyset$ then $\mathrm{occ}(Y) = 0 \leq U_K(X)$. Otherwise let $\ell = |Y|$. By Lemma 2, every $q \in T(Y)$ belongs to $T(X)$ and satisfies $|t_q| \geq \ell$. Hence

$$\mathrm{occ}(Y) = \ell \sum_{q \in T(Y)} \frac{1}{|t_q|} \leq \ell \sum_{q \in T(X),\ |t_q| \geq \ell} \frac{1}{|t_q|} = \ell \cdot W_\ell(X). \tag{5}$$

The set $\{\lambda \in \Lambda(X) : \lambda \geq \ell\}$ is not empty, because it contains the length of any transaction in $T(Y)$. Let λ^* be its minimum. No transaction in $T(X)$ has a length in $[\ell, \lambda^*)$, so $W_\ell(X) = W_{\lambda^*}(X)$. Moreover $\ell \leq \lambda^*$ and $\ell \leq K$, so $\ell \leq \min(\lambda^*, K)$. Together with (5),

$$\mathrm{occ}(Y) \leq \min(\lambda^*, K) \cdot W_{\lambda^*}(X) \leq U_K(X).$$

□

Two special cases are known. Without a size limit ($K \geq \max \Lambda(X)$), the bound becomes

$$\mathrm{UBO}(X) = \max_{\lambda \in \Lambda(X)} \lambda \cdot W_\lambda(X), \tag{6}$$

which depends only on transaction lengths. This is the upper bound on occupancy used by HEP and DFHOI [7, 8]; we refer to it as the transaction-length bound. Ignoring transaction lengths gives the purely residual bound $K \cdot S(X)$, which follows from $|Y| \leq K$ and $T(Y) \subseteq T(X)$ alone.

Proposition 4 (Dominance) *For every itemset X and integer $K \geq 1$, $U_K(X) \leq \min\{\mathrm{UBO}(X),\ K \cdot S(X)\}$, and $U_K(X)$ is non-decreasing in K.*

Proof For every $\lambda \in \Lambda(X)$, $\min(\lambda, K) \cdot W_\lambda(X) \leq \lambda \cdot W_\lambda(X) \leq \mathrm{UBO}(X)$, and $\min(\lambda, K) \cdot W_\lambda(X) \leq K \cdot W_\lambda(X) \leq K \cdot S(X)$ because $W_\lambda(X) \leq W_1(X) = S(X)$. Taking the maximum over λ gives the first claim. Each term $\min(\lambda, K) \cdot W_\lambda(X)$ is non-decreasing in K, and so is their maximum. □

The bound is strictly tighter than both special cases in general.

Example 2 In Example 1, consider $X = \{C, E\}$ with $T(X) = \{4, 5\}$, so $\Lambda(X) = \{4\}$, $W_4(X) = 1/2$ and $\mathrm{supp}(X) = 2 \geq \sigma$. The transaction-length bound is $\mathrm{UBO}(X) = 4 \cdot 1/2 = 2 \geq \xi = 7/4$, so it cannot prune X. In the search of Section 5, $\{C, E\}$ is generated in the class of prefix $\{C\}$, whose other member is $\{C, D\}$; the largest itemset that can still contain $\{C, E\}$ there is $\{C, D, E\}$, so $K = 3$. Then $U_3(X) = \min(4, 3) \cdot 1/2 = 3/2 < 7/4$, and X is pruned safely: indeed $\mathrm{occ}(\{C, E\}) = 1$ and $\mathrm{occ}(\{C, D, E\}) = 3/2$.

The next result explains why the transaction-length bound fails on databases whose transactions have equal length.

Proposition 5 (Uniform lengths) *If every transaction has length L, then for every itemset X with $T(X) \neq \emptyset$*

$$\mathrm{UBO}(X) = \mathrm{supp}(X) \qquad \textit{and} \qquad U_K(X) = \frac{\min(L, K)}{L} \cdot \mathrm{supp}(X).$$

Proof Here $\Lambda(X) = \{L\}$ and $W_L(X) = \mathrm{supp}(X)/L$, so the maximum in (4) has a single term, $\min(L, K) \cdot \mathrm{supp}(X)/L$, which equals $\mathrm{supp}(X)$ when $K \geq L$. □

Since every itemset reached by a miner already satisfies $\mathrm{supp}(X) \geq \sigma \geq \xi$, the transaction-length bound never prunes on such databases, whereas $U_K(X) < \xi$ holds as soon as $K < L\xi/\mathrm{supp}(X)$.

Computation.

Before mining, we re-index the transactions in non-decreasing order of length, as DFHOI does [8], so that sorted TID-lists are also sorted by transaction length. For a TID-list sorted in this way, a single backward scan accumulates $W_\lambda(X)$ for all $\lambda \in \Lambda(X)$ in decreasing order of λ, and the maximum in (4) is updated at each change of length. The bound therefore costs $O(|T(X)|)$ and is computed in the pass that also produces $S(X)$. With uniform lengths, Proposition 5 gives it in $O(1)$. The purely residual bound $K \cdot S(X)$ costs $O(1)$ once $S(X)$ is known.

5 The AURA-HOI algorithm

5.1 Search space

We fix a total order on the items. As in Eclat [5], the search is organized in prefix equivalence classes. A class is an ordered list $E = \langle N_0, \ldots, N_{k-1} \rangle$ of nodes that share a prefix P; node N_i stores the itemset $X_i = P \cup \{x_i\}$, its TID-list $T(X_i)$ in increasing order and its reciprocal sum $S(X_i)$. The children of N_i form the class of prefix X_i, whose candidates are $X_i \cup \{x_j\}$ for $j > i$, with $T(X_i \cup \{x_j\}) = T(X_i) \cap T(X_j)$. Without pruning, this enumeration generates every non-empty itemset exactly once [5].

For node N_i we call $r_i = k - 1 - i$ its residual length: the number of items that can still be appended to X_i in its own subtree. Every itemset generated in the subtree of N_i has at most $|X_i| + r_i$ items.

5.2 Pruning rules

Let $E = \langle N_0, \ldots, N_{k-1} \rangle$ be a class, $X = X_i$ the node being expanded, and $Y = X \cup \{x_j\}$ $(j > i)$ a candidate child. AURA-HOI applies three rules.

R1 (support) Discard Y if $\mathrm{supp}(Y) < \sigma$.
R2 (node bound) Discard Y if $U_K(Y) < \xi$ with $K = |Y| + r_i - 1$.
R3 (class bound) Do not expand X if $(|X| + r_i) \cdot S(X) < \xi$. After the children of X have been generated, do not recurse into them if $(|X| + c) \cdot S(X) < \xi$, where c is the number of children that survived R1 and R2.

Rules R1 and R3 cost $O(1)$. When all transactions have the same length L, AURA-HOI evaluates R2 and R3 with the closed form of Proposition 5: the value $(|X| + K') \cdot S(X)$ in R3 is replaced by $U_{|X|+K'}(X) = \min(L, |X| + K') \cdot \mathrm{supp}(X)/L$ for $K' \in \{r_i, c\}$, which is never larger by Proposition 4.

Rule R2 deserves care. A discarded child is removed from the class of prefix X altogether. It is therefore lost not only as the root of its own subtree but also as an item that the *earlier* children of X could append. The itemsets that contain Y and could still be generated in the subtree of X have the form $X \cup \{x_j\} \cup Z$ with $Z \subseteq \{x_l : l > i,\ l \neq j\}$, and there are $r_i - 1$ such items x_l. This is why R2 uses $K = |Y| + r_i - 1$, which counts all other children of X, and not only the items after x_j.

Remark 1 (A tempting but unsound shortcut) Using $K = |Y| + (k - 1 - j)$ in R2, that is, counting only the items that follow x_j, loses HOIs. Let $D = \langle \{A, B, C\}, \{A\} \rangle$ and $\alpha = 2/5$, so

Algorithm 1 AURA-HOI

Require: database D, threshold $\alpha \in (0, 1]$
Ensure: the set $\mathcal{H}$ of all HOIs

$\xi \leftarrow \alpha n$; $\sigma \leftarrow \lceil \xi \rceil$; re-index transactions by non-decreasing length
$E_0 \leftarrow \langle$ nodes $\{x\}$ with $\mathrm{supp}(\{x\}) \geq \sigma$ and $\mathrm{UBO}(\{x\}) \geq \xi \rangle$ ▷ R1 and Theorem 3
emit every $\{x\} \in E_0$ with $\mathrm{occ}(\{x\}) \geq \xi$; $\textsc{Mine}(E_0)$
procedure $\textsc{Mine}(E = \langle N_0, \ldots, N_{k-1} \rangle)$
    for $i = 0, \ldots, k-2$ **do**
        $X \leftarrow X_i$; $r \leftarrow k - 1 - i$
        if $(|X| + r) \cdot S(X) < \xi$ **then continue** ▷ R3
        $m \leftarrow \textsc{ArenaMark}$; $C \leftarrow \langle\,\rangle$
        for $j = i+1, \ldots, k-1$ **do**
            $T \leftarrow T(X) \cap T(X_j)$ ▷ TID-list stored in the arena
            if $|T| < \sigma$ **then continue** ▷ R1
            compute $S(Y)$ and $U_{|Y|+r-1}(Y)$ for $Y = X \cup \{x_j\}$ in one pass over T
            if $U_{|Y|+r-1}(Y) < \xi$ **then continue** ▷ R2
            if $|Y| \cdot S(Y) \geq \xi$ **then** emit Y
            append the node of Y to C
        if $|C| \geq 2$ **and** $(|X| + |C|) \cdot S(X) \geq \xi$ **then** $\textsc{Mine}(C)$ ▷ R3
        $\textsc{ArenaRewind}(m)$

that $\xi = 4/5$ and $\sigma = 1$. The itemset $\{A, B, C\}$ is an HOI, since $\mathrm{occ}(\{A, B, C\}) = 3 \cdot 1/3 = 1$. In the class of prefix $\{A\}$, the candidate $Y = \{A, C\}$ comes last, so the shortcut uses $K = 2$ and obtains $U_2(Y) = \min(3, 2) \cdot 1/3 = 2/3 < \xi$. It discards $\{A, C\}$, and $\{A, B, C\}$, which would be generated from $\{A, B\}$ and $\{A, C\}$, is never produced. Rule R2 uses $K = 3$ instead, obtains $U_3(Y) = 1 \geq \xi$, keeps $\{A, C\}$ and finds $\{A, B, C\}$.

5.3 Correctness

Theorem 6 (Correctness and completeness) *Algorithm 1 emits every HOI exactly once and emits nothing else.*

Proof Soundness. An itemset is emitted only after the exact test $|Y| \cdot S(Y) \geq \xi$, that is, $\mathrm{occ}(Y) \geq \xi$.

Uniqueness. The prefix-class enumeration generates each itemset at most once [5], and pruning only removes itemsets from it.

Completeness. Let H be an HOI. Without pruning, the enumeration generates H along a unique path of nodes $X^{(1)} \subset X^{(2)} \subset \cdots \subset X^{(|H|)} = H$, where $X^{(s)}$ consists of the first s items of H in the item order. We show that no rule removes a node of this path or blocks the recursion along it. Every node on the path is a subset of H.

- R1: by Proposition 1 and Lemma 2(i), $\mathrm{supp}(X^{(s)}) \geq \mathrm{supp}(H) \geq \sigma$.

- Root filter of Algorithm 1: $X^{(1)} \subseteq H$, so $\mathrm{UBO}(X^{(1)}) \geq \mathrm{occ}(H) \geq \xi$ by Theorem 3 with an unbounded K.
- R2: suppose $Y = X^{(s+1)}$ is generated as a child of $X = X^{(s)} = X_i$ in a class of size k. The itemset H is generated in the subtree of X, so $H = X \cup Z'$ with Z' made of x_j and of other members x_l ($l > i$) of the class, and hence $|H| \leq |X| + r_i = |Y| + r_i - 1$. By Theorem 3, $U_{|Y|+r_i-1}(Y) \geq \mathrm{occ}(H) \geq \xi$, so R2 keeps Y. Rule R2 is also applied to the other members of the class that contain an item of H, that is, to the nodes $X \cup \{x_l\}$ with $x_l \in H$; each of them is a subset of H and the same argument shows that it is kept, so all items of $H \setminus X$ remain available.
- R3, first test: when $X = X^{(s)}$ with $s < |H|$ is expanded, H lies in its subtree, so $|H| \leq |X| + r_i$. By Theorem 3, $U_{|X|+r_i}(X) \geq \mathrm{occ}(H) \geq \xi$, and by Proposition 4, $(|X| + r_i) \cdot S(X) \geq U_{|X|+r_i}(X)$; so neither form of the test prunes X.
- R3, second test: when $s + 1 < |H|$, the items of $H \setminus X$ are among the c surviving children of X, so $|H| \leq |X| + c$, and the same argument gives $(|X| + c) \cdot S(X) \geq \xi$. If $s + 1 = |H|$, then H is a child of X and is emitted when it is generated, before this test.

Hence H is generated and, being an HOI, emitted. □

5.4 Memory management and complexity

The TID-lists of the children of X are needed only while the subtree of X is mined. AURA-HOI therefore allocates them from a contiguous arena: it records the current offset before generating the children (ArenaMark) and restores it afterwards (ArenaRewind). This replaces one heap allocation and one release per TID-list by a pointer increment. The small array of node headers of each class is still allocated on the heap. At any time the arena holds the TID-lists of the classes on the current path of the search, that is, at most $d \cdot k_{\max}$ lists of at most n identifiers each, where d is the depth of the path and $k_{\max}$ the largest class size. This is the same asymptotic space as any depth-first TID-list miner; the arena reduces allocation cost and fragmentation, not the asymptotic space.

Processing a candidate $Y = X \cup \{x_j\}$ costs $O(|T(X)| + |T(X_j)|)$ for the merge-based intersection plus $O(|T(Y)|)$ for $S(Y)$ and $U_K(Y)$, which are computed in one pass over the result. Rule R3 costs $O(1)$ per node. The number of candidates depends on the data and on how much the rules prune, which Section 6 measures.

6 Experimental evaluation

6.1 Setup

Implementations.

HEP, DFHOI and AURA-HOI are implemented in C in the same code base, share the same data loader and transaction re-indexing, and were compiled with GCC 13.3 at optimization level `-O2`. HEP and DFHOI are our implementations of the published algorithms [7, 8]: HEP generates candidates level by level and DFHOI depth-first, and both prune with the transaction-length bound (6) and the support constraint. All three compare occupancies with the threshold using the same relative tolerance of

10^{-9}, so that itemsets whose occupancy equals ξ exactly are not lost to floating-point rounding (for instance, 0.1×19 evaluates to 1.9000000000000001 in double precision). The threshold $\sigma = \lceil \alpha n \rceil$ is computed with rational arithmetic.

Platform and protocol.

All runs used one thread on an Intel Core i7-10700 (2.9 GHz base, 16 MB L3 cache) with 32 GB of RAM under Ubuntu Linux 24.04 (kernel 6.8). HEP and DFHOI are sequential algorithms, so a single-threaded comparison measures algorithmic efficiency rather than parallel hardware. Runtime is the duration of the mining phase; loading the data is excluded. No time limit was imposed: every run was allowed to complete. To protect the machine, each run was limited to 20 GB of address space; a run that exceeds this limit is reported as failed, and for that algorithm the smaller thresholds on the same dataset are not run, because every pruning rule of the three algorithms compares a bound with ξ, so a smaller threshold can only enlarge the search. A configuration was repeated five times if its first run took less than 10 s, three times if it took less than 120 s and once otherwise; we report the median.

Memory.

All three programs share the same executable, whose resident size after loading the data is dominated by linked libraries and is identical for every algorithm. We therefore report the memory attributable to mining: just before the mining phase, we record the resident set size and reset the kernel's peak-resident-size counter (`VmHWM`) by writing `5` to `/proc/self/clear_refs` [31]; the reported value is the peak resident size during mining minus the recorded value.

Search effort.

We count candidate intersections, that is, the number of times an algorithm intersects two TID-lists to form a candidate. This is the unit of work of all three algorithms, and it is the quantity that the bounds of Section 4 reduce.

6.2 Datasets

Table 2 lists the datasets, all of which are public benchmarks distributed by the FIMI repository [32] and the SPMF library [33, 34]. Four are dense and five are sparse. `mushrooms`, `chess` and `connect` have transactions of a single length, which is the case covered by Proposition 5. For each dataset we chose a grid of thresholds that ranges from settings with few or no HOIs to settings with many HOIs; ?? of the ?? configurations have at least one HOI. The dataset `pumsb` is discussed separately in Section 6.8.

6.3 Correctness

We verified the three implementations against an exact oracle. For ?? random databases with 4 to 11 items and 5 to 30 transactions, half with transactions of varying length and half with transactions of a single length, and for $\alpha \in \{0.1, 0.2, 0.3, 0.4\}$,

Table 2 Datasets. Length gives the minimum and maximum transaction length (a single value means that all transactions have the same length); density is the average length divided by the number of items.

Dataset	Type	n	Items	Avg. len.	Length	Density (%)	$\#\alpha$	α range
??	??	??	??	??	??	??	??	??

Table 3 Summary per dataset. Finished: configurations completed within the limits. Faster: configurations in which AURA-HOI is faster than DFHOI, among those completed by both. Speedup: runtime of DFHOI divided by that of AURA-HOI (geometric mean and maximum) over the same configurations. Reduction: median reduction of candidate intersections relative to DFHOI.

		Finished				Speedup vs. DFHOI		Reduction
Dataset	Configs	HEP	DFHOI	AURA-HOI	Faster	Geo. mean	Max	(%)
??	??	??	??	??	??	??	??	??

placeholder: runtime

Fig. 1 Runtime (seconds, logarithmic scale) as a function of the threshold α. Missing points exceeded the memory limit.

the oracle enumerates every itemset and evaluates its occupancy in rational arithmetic. In all ?? cases, HEP, DFHOI, AURA-HOI and each of its ablation variants (Section 6.6) returned exactly the oracle's number of HOIs (mismatches: ??). On the benchmark datasets, whenever at least two algorithms completed a configuration (?? configurations), they returned the same number of HOIs (mismatches: ??).

6.4 Runtime and search effort

Table 3 summarizes the comparison per dataset, Figures 1 and 2 show runtime and search effort for every threshold, and Table A1 in the Appendix gives all values.

[Runtime discussion: to be written from the final results.]

6.5 Memory

[Memory discussion: to be written from the final results.]

Fig. 2 Candidate intersections (logarithmic scale) as a function of the threshold α.

Fig. 3 Peak memory attributable to mining (MB, logarithmic scale).

Table 4 Ablation study: runtime in seconds and, in parentheses, candidate intersections. ML: memory limit exceeded.

Dataset	α	Full	No tail cap	No class bound	No residual	No arena
??	??	??	??	??	??	??

6.6 Ablation study

To measure the contribution of each component, we disabled them one at a time: *no tail cap* uses the transaction-length bound (6) instead of U_K in rule R2; *no class bound* removes rule R3; *no residual* does both, so that only transaction lengths are used, as in DFHOI; and *no arena* allocates every TID-list on the heap. Table 4 reports runtime and candidate intersections.

[Ablation discussion: to be written from the final results.]

6.7 Scalability

[Scalability discussion: to be written from the final results.]

6.8 Limitations

[Limitations (pumsb, sparse data): to be written from the final results.]

placeholder: scalability

Fig. 4 Runtime on growing prefixes of the database at a fixed threshold.

7 Conclusion

[Conclusion: to be written from the final results.]

Acknowledgements. Not applicable.

Declarations

- **Funding:** Funding not applicable. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.
- **Competing interests:** The author declares no competing interests.
- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** All datasets are publicly available from the FIMI repository [32] and the SPMF library [33].
- **Materials availability:** Not applicable.
- **Code availability:** The source code of the three implementations and the scripts that reproduce every experiment are available from the corresponding author on reasonable request.
- **Author contribution:** Q. Van conceived the method, proved the results, implemented the algorithms, performed the experiments and wrote the manuscript.

Appendix A Results for every configuration

Table A1: Runtime (s; median of the repetitions; fastest in bold), candidate intersections and peak memory attributable to mining (MB) for every configuration. ML: memory limit exceeded; ML*: not run because the same algorithm exceeded the memory limit at a larger threshold.

			Runtime (s)			Intersections			Memory (MB)		
Dataset	α	HOIs	HEP	DFHOI	AURA-HOI	DFHOI	AURA-HOI	HEP	DFHOI	AURA-HOI	
??	??	??	??	??	??	??	??	??	??	??	

References

[1] Agrawal, R., Imieliński, T., Swami, A.: Mining association rules between sets of items in large databases. In: Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data, pp. 207–216. ACM, ??? (1993). https://doi.org/10.1145/170035.170072

[2] Agrawal, R., Srikant, R.: Fast algorithms for mining association rules. In: Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), pp. 487–499 (1994). https://www.vldb.org/conf/1994/P487.PDF

[3] Han, J., Pei, J., Yin, Y.: Mining frequent patterns without candidate generation. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, pp. 1–12. ACM, ??? (2000). https://doi.org/10.1145/342009.335372

[4] Zaki, M.J., Parthasarathy, S., Ogihara, M., Li, W.: New algorithms for fast discovery of association rules. In: Proceedings of the Third International Conference on Knowledge Discovery and Data Mining (KDD), pp. 283–286. AAAI Press, ??? (1997). https://aaai.org/papers/kdd97-060-new-algorithms-for-fast-discovery-of-association-rules/

[5] Zaki, M.J.: Scalable algorithms for association mining. IEEE Transactions on Knowledge and Data Engineering **12**(3), 372–390 (2000) https://doi.org/10.1109/69.846291

[6] Tang, L., Zhang, L., Luo, P., Wang, M.: Incorporating occupancy into frequent pattern mining for high quality pattern recommendation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM), pp. 75–84. ACM, ??? (2012). https://doi.org/10.1145/2396761.2396775

[7] Deng, Z.-H.: Mining high occupancy itemsets. Future Generation Computer Systems **102**, 222–229 (2020) https://doi.org/10.1016/j.future.2019.07.039

[8] Nguyen, L.T.T., Mai, T., Pham, G.-H., Yun, U., Vo, B.: An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. Knowledge-Based Systems **267**, 110441 (2023) https://doi.org/10.1016/j.knosys.2023.110441

[9] Datta, S., Mali, K., Ghosh, U.: High occupancy itemset mining with consideration of transaction occupancy. Arabian Journal for Science and Engineering **47**(2), 2061–2075 (2022) https://doi.org/10.1007/s13369-021-06075-8

[10] Kim, H., Ryu, T., Lee, C., Kim, H., Truong, T., Fournier-Viger, P., Pedrycz, W., Yun, U.: Mining high occupancy patterns to analyze incremental data in intelligent systems. ISA Transactions **131**, 460–475 (2022) https://doi.org/10.1016/j.isatra.2022.05.003

[11] Tran, T.-N., Truong Hoang, V., Truong, T.-C., Voznak, M.: A hierarchical set-enumeration tree enabling high occupancy item set mining and the use of an adaptive occupancy threshold. Applied Intelligence **55**(3), 205 (2025) https://doi.org/10.1007/s10489-024-06166-7

[12] Yıldırım, İ.: Mining top-k high occupancy itemsets. Black Sea Journal of Engineering and Science **8**(6), 1723–1730 (2025) https://doi.org/10.34248/bsengineering.1744061

[13] Gan, W., Lin, J.C.-W., Fournier-Viger, P., Chao, H.-C., Yu, P.S.: HUOPM: High-utility occupancy pattern mining. IEEE Transactions on Cybernetics **50**(3), 1195–1208 (2020) https://doi.org/10.1109/TCYB.2019.2896267

[14] Kumar, M.J.K., Rana, D.: HAUOPM: High average utility occupancy pattern mining. Arabian Journal for Science and Engineering **49**(3), 3397–3416 (2024) https://doi.org/10.1007/s13369-023-07971-x

[15] Cho, M., Kim, H., Park, S., Kim, D., Kim, D., Yun, U.: Advanced approach for mining utility occupancy patterns in incremental environment. Knowledge-Based Systems **306**, 112713 (2024) https://doi.org/10.1016/j.knosys.2024.112713

[16] Liu, M., Qu, J.: Mining high utility itemsets without candidate generation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM), pp. 55–64. ACM, ??? (2012). https://doi.org/10.1145/2396761.2396773

[17] Zida, S., Fournier-Viger, P., Lin, J.C.-W., Wu, C.-W., Tseng, V.S.: EFIM: A fast and memory efficient algorithm for high-utility itemset mining. Knowledge and Information Systems **51**(2), 595–625 (2017) https://doi.org/10.1007/s10115-016-0986-0

[18] Zaki, M.J., Gouda, K.: Fast vertical mining using diffsets. In: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 326–335. ACM, ??? (2003). https://doi.org/10.1145/956750.956788

[19] Deng, Z., Wang, Z., Jiang, J.: A new algorithm for fast mining frequent itemsets using N-lists. Science China Information Sciences **55**(9), 2008–2030 (2012) https://doi.org/10.1007/s11432-012-4638-z

[20] Deng, Z.-H., Lv, S.-L.: PrePost+: An efficient N-lists-based algorithm for mining frequent itemsets via Children–Parent equivalence pruning. Expert Systems with Applications **42**(13), 5424–5432 (2015) https://doi.org/10.1016/j.eswa.2015.03.004

[21] Pasquier, N., Bastide, Y., Taouil, R., Lakhal, L.: Discovering frequent closed itemsets for association rules. In: Database Theory – ICDT'99. Lecture Notes in

Computer Science, vol. 1540, pp. 398–416. Springer, ??? (1999). https://doi.org/10.1007/3-540-49257-7_25

[22] Zaki, M.J., Hsiao, C.-J.: CHARM: An efficient algorithm for closed itemset mining. In: Proceedings of the 2002 SIAM International Conference on Data Mining, pp. 457–473. SIAM, ??? (2002). https://doi.org/10.1137/1.9781611972726.27

[23] Uno, T., Kiyomi, M., Arimura, H.: LCM ver. 2: Efficient mining algorithms for frequent/closed/maximal itemsets. In: Proceedings of the IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI'04). CEUR Workshop Proceedings, vol. 126 (2004). https://ceur-ws.org/Vol-126/uno.pdf

[24] Bayardo, R.J.: Efficiently mining long patterns from databases. ACM SIGMOD Record **27**(2), 85–93 (1998) https://doi.org/10.1145/276305.276313

[25] Aggarwal, C.C., Han, J. (eds.): Frequent Pattern Mining. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-07821-2

[26] Fournier-Viger, P., Lin, J.C.-W., Vo, B., Chi, T.T., Zhang, J., Le, H.B.: A survey of itemset mining. WIREs Data Mining and Knowledge Discovery **7**(4), 1207 (2017) https://doi.org/10.1002/widm.1207

[27] Luna, J.M., Fournier-Viger, P., Ventura, S.: Frequent itemset mining: A 25 years review. WIREs Data Mining and Knowledge Discovery **9**(6), 1329 (2019) https://doi.org/10.1002/widm.1329

[28] Fournier-Viger, P., Lin, J.C.-W., Truong-Chi, T., Nkambou, R.: A survey of high utility itemset mining. In: High-Utility Pattern Mining, pp. 1–45. Springer, ??? (2019). https://doi.org/10.1007/978-3-030-04921-8_1

[29] Gan, W., Lin, J.C.-W., Fournier-Viger, P., Chao, H.-C., Tseng, V.S., Yu, P.S.: A survey of utility-oriented pattern mining. IEEE Transactions on Knowledge and Data Engineering **33**(4), 1306–1327 (2021) https://doi.org/10.1109/TKDE.2019.2942594

[30] Fournier-Viger, P., Wu, C.-W., Zida, S., Tseng, V.S.: FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: Foundations of Intelligent Systems (ISMIS 2014). Lecture Notes in Computer Science, vol. 8502, pp. 83–92. Springer, ??? (2014). https://doi.org/10.1007/978-3-319-08326-1_9

[31] The Linux Kernel Documentation: The /proc Filesystem. https://www.kernel.org/doc/html/latest/filesystems/proc.html. Accessed 14 October 2026 (2026)

[32] Goethals, B., Zaki, M.J.: Frequent Itemset Mining Dataset Repository. http://fimi.uantwerpen.be/. Accessed 14 October 2026 (2004)

[33] Fournier-Viger, P.: SPMF: An Open-Source Data Mining Library – Datasets.

https://www.philippe-fournier-viger.com/spmf/index.php?link=datasets.php. Accessed 14 October 2026 (2026)

[34] Fournier-Viger, P., Lin, J.C.-W., Gomariz, A., Gueniche, T., Soltani, A., Deng, Z., Lam, H.T.: The SPMF open-source data mining library version 2. In: Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2016). Lecture Notes in Computer Science, vol. 9853, pp. 36–40. Springer, ??? (2016). https://doi.org/10.1007/978-3-319-46131-1_8